

Satellite Change Detection Project Handoff Guide

1. What This Project Is

This project builds a satellite image change detection pipeline on the LEVIR-CD dataset. The goal is to compare two satellite images of the same area taken at different times and predict which pixels correspond to meaningful structural change, especially new or removed buildings.

The work is organized as a staged improvement pipeline:

1. Baseline Siamese U-Net with contrastive loss
2. Threshold sweep for better baseline evaluation
3. BCE-Dice segmentation upgrade with a lightweight change head
4. ResNet-18 backbone upgrade
5. CBAM attention upgrade
6. RAPIDS/cuML acceleration steps
7. Dask sliding-window inference on full-resolution images
8. Final comparison and reporting

The most important modeling insight is that the original baseline learned embedding distances, while evaluation was based on binary segmentation quality. That mismatch is why the BCE-Dice and change-head upgrade is central to the project.

2. Dataset and Environment

Dataset

- Dataset: LEVIR-CD
- Splits:
 - Train: 445 pairs
 - Validation: 64 pairs
 - Test: 128 pairs
- Original image size: 1024 x 1024
- Working crop size in the notebooks: 256 x 256

Expected Colab Environment

- Platform: Google Colab
- Preferred runtime: GPU
- Typical GPU: T4
- Dataset extraction location:
- `/content/levir_data``
- Repo clone location:
- `/content/Satellite-Change-thingy-with-U-Net``

Important Colab Notes

- Colab storage under `/content`` is temporary
- Trained checkpoints are lost if the runtime resets
- Kaggle dataset files must be downloaded again if the runtime restarts
- GPU limits in free Colab are dynamic and not published exactly by Google

3. What Has Been Added to GitHub

The repository was turned from an almost empty Git folder into a structured ML project.

Root Files

`README.md``

Main project overview for GitHub. It includes:

- project description
- architecture summary
- quickstart steps
- local setup
- dataset setup
- command-line usage
- project structure
- roadmap

- references

This is the first file a teammate or evaluator should read.

`requirements.txt`

Main Python dependency list for local runs and Colab package installation.

`requirements\_rapids.txt`

Additional GPU-native RAPIDS packages for the acceleration phase:

- `cudf-cu11`
- `cuml-cu11`
- `cucim`
- `dask-cuda`

`.gitignore`

Prevents temporary files, model weights, notebook cache, and generated outputs from being committed by mistake.

`.python-version`

Pins Python 3.10 as the expected project version.

`setup.py`

Allows the repository to behave like an installable package.

`LICENSE`

MIT license file with the team names.

4. Config Files

The project stores training and inference settings in YAML rather than hardcoding everything inside Python files.

``configs/baseline.yaml``

This stores the original baseline configuration:

- custom Siamese U-Net
- contrastive loss
- batch size
- learning rate
- cosine scheduler
- threshold sweep settings

Use this for the baseline and for reproducing the original low-performing setup.

``configs/full.yaml``

This stores the intended final stronger configuration:

- ResNet-18 backbone
- CBAM enabled
- BCE-Dice loss
- staged fine-tuning
- sliding-window inference defaults

Use this for the upgraded model pipeline.

5. Source Code Structure

All reusable Python code lives inside ``src/``.

``src/__init__.py``

Marks ``src`` as a Python package.

``src/utils.py``

General helper utilities, including:

- seeding

- device selection
- metric calculations
- parameter counting
- plotting helpers
- Gaussian weight generation for sliding-window inference
- qualitative visualization helpers

``src/losses.py``

Contains the loss functions:

- ``ContrastiveLoss``
- ``BCEDiceLoss``

This file is important because it contains both the original objective and the improved segmentation-aligned objective.

``src/factory.py``

Shared constructors for:

- loading YAML configs
- building models
- building losses
- building optimizers
- building schedulers

This keeps the repo modular and reduces repeated setup code.

``src/dataset.py``

Contains:

- dataset root auto-detection
- LEVIR-CD dataset class
- synchronized paired transforms
- optional cuCIM loading path
- dataloader creation helpers

- positive class weight estimation

This file is essential for keeping image A, image B, and the mask aligned.

`src/train.py`

Contains the main training pipeline:

- training loop
- validation calls
- checkpoint saving
- history tracking
- staged backbone freezing and unfreezing
- CLI entrypoint

This is the file used by command-line training and by notebook helper calls.

`src/evaluate.py`

Contains:

- validation/test evaluation
- threshold sweep logic
- plotting for threshold and PR curves
- metric conversion from TP/FP/FN

This file completed the originally incomplete threshold sweep logic.

`src/inference.py`

Contains:

- single-tile inference
- sliding-window inference
- Gaussian overlap blending
- Dask-based parallel tile execution
- CLI inference entrypoint

This file is used for full-resolution 1024 x 1024 inference.

6. Model Files

The repo separates model components inside ``src/models/``.

``src/models/cbam.py``

Implements CBAM attention:

- channel attention
- spatial attention
- combined CBAM block

Used in the upgraded decoder to sharpen skip features.

``src/models/siamese_unet.py``

Contains the original baseline model components:

- custom encoder
- custom decoder
- shared-weight Siamese U-Net
- lightweight ``ChangeHead``

This file supports both:

- the old baseline behavior
- the Step 2 BCE-Dice upgrade using the same backbone

``src/models/siamese_resnet.py``

Contains the upgraded architecture:

- shared pretrained ResNet-18 encoder
- ResNet-compatible decoder
- optional CBAM skip attention
- final change head

This is the main final model family.

7. Notebook Files

The notebooks are organized as a teaching and execution path.

``notebooks/01_setup_and_data.ipynb``

Purpose:

- environment setup
- package installation
- dataset download
- dataloader sanity checking

Run this first in a new Colab session.

``notebooks/02_baseline_model.ipynb``

Purpose:

- reproduce baseline custom encoder model
- train with contrastive loss
- establish weak baseline metrics

Expected behavior:

- low segmentation quality
- poor or near-zero test F1 at default threshold

``notebooks/03_bce_dice_upgrade.ipynb``

Purpose:

- complete threshold sweep
- recover best validation threshold for baseline
- visualize baseline predictions
- train the custom encoder with BCE-Dice and change head

This is the first major improvement notebook.

``notebooks/04_resnet_backbone.ipynb``

Purpose:

- replace custom encoder with ResNet-18
- freeze backbone initially
- unfreeze later with lower backbone LR

``notebooks/05_cbam_attention.ipynb``

Purpose:

- enable CBAM in the decoder
- retrain the ResNet-based model

``notebooks/06_rapids_pipeline.ipynb``

Purpose:

- optional RAPIDS installation
- cuCIM loading benchmark
- GPU-native metric acceleration hooks
- CPU vs GPU timing table

``notebooks/07_dask_inference.ipynb``

Purpose:

- sliding-window inference
- Dask parallel tile execution
- full-resolution qualitative results

``notebooks/08_full_pipeline.ipynb``

Purpose:

- combined master notebook
- high-level report notebook

- final comparison and summary cells

8. Results and Progress So Far

The following concrete progress has already been achieved during execution.

Baseline Training

- Baseline notebook completed
- Best checkpoint path:
- ``results/baseline/checkpoints/siamese_baseline_best.pth``

Completed Threshold Sweep

Best threshold found on validation:

- ``BEST_THRESHOLD = 0.2775510251522064``

Best baseline sweep metrics:

- F1: ``0.2394``
- Precision: ``0.1971``
- Recall: ``0.3049``

This is much better than using threshold 1.0 on the baseline.

Important Practical Fixes Found During Execution

The following runtime issues were discovered and fixed during Colab execution:

1. ``num_workers = 2`` caused repeated dataloader worker shutdown errors in Colab
2. Setting ``num_workers = 0`` is safer for Colab
3. The qualitative visualization cell originally ran without ``torch.no_grad()``
4. That caused GPU out-of-memory during inference visualization
5. Colab runtime resets delete checkpoints and dataset files under ``/content``
6. Notebook 3 originally expected the baseline checkpoint at the wrong path

These are important teammate instructions, not just optional notes.

9. Exact Colab Workflow for a Teammate

This is the recommended teammate workflow.

Initial Setup

1. Open Google Colab
2. Connect a GPU runtime
3. Clone the repo
4. Install requirements
5. Upload ``kaggle.json``
6. Move ``kaggle.json`` to ``/root/.kaggle/kaggle.json``
7. Download and unzip the LEVIR-CD dataset
8. Set the repo root in Python with:
 - ``os.chdir("/content/Satellite-Change-thingy-with-U-Net")``
 - add repo path to ``sys.path``

Run Order

Recommended full notebook order:

1. ``01_setup_and_data.ipynb``
2. ``02_baseline_model.ipynb``
3. ``03_bce_dice_upgrade.ipynb``
4. ``04_resnet_backbone.ipynb``
5. ``05_cbam_attention.ipynb``
6. ``06_rapids_pipeline.ipynb``
7. ``07_dask_inference.ipynb``
8. ``08_full_pipeline.ipynb``

Important Execution Rules

For Colab stability:

- set ``config["data"]["num_workers"] = 0``
- use ``torch.no_grad()`` in visualization-only inference cells

- save or copy important checkpoints before experimenting further
- if runtime resets, redownload dataset and recreate any lost checkpoints

10. Exact Resume Point for the Team

If someone resumes after the current stopping point, they do not need to start from zero conceptually.

Current logical state:

- baseline complete
- threshold sweep complete
- baseline visualizations complete
- Step 2 training was started but interrupted by Colab GPU quota issues

Therefore the next meaningful modeling step is:

Resume from Step 2

Run the BCE-Dice + change-head training cell with:

- dataset root set to ``/content/levir_data``
- ``num_workers = 0``

After Step 2 finishes, continue with:

1. Step 3 ResNet-18 notebook
2. Step 4 CBAM notebook
3. Step 6 sliding-window inference notebook
4. Step 8 final reporting notebook

11. Why This Structure Matters

This repo is now much easier for a teammate to understand because:

- configuration is separated from logic
- dataset handling is centralized
- baseline and upgraded models are separated cleanly
- training and evaluation are reusable

- inference and deployment-style logic are isolated
- notebooks are organized by improvement stage instead of being one giant file

That means a teammate can either:

- follow the notebooks as a course/project submission flow, or
- use the Python package directly for clean training and evaluation

12. Recommended Next Changes for GitHub

Before the next teammate run, the repo should ideally be updated with the execution fixes discovered during real Colab use:

1. set notebook training configs to ``num_workers = 0``
2. update notebook checkpoint-loading paths where needed
3. wrap qualitative inference cells in ``torch.no_grad()``
4. note in the README that Colab runtime resets delete local checkpoints
5. optionally add a short "resume after reset" section to the README

These are practical quality-of-life fixes that reduce teammate confusion.

13. Final Summary

This project is a structured satellite change detection pipeline built for LEVIR-CD using Siamese deep learning models. The GitHub repo now contains:

- organized source code
- configs
- staged notebooks
- packaging files
- documentation

Execution so far confirms:

- the baseline is weak
- threshold sweep improves the baseline evaluation
- the next critical step is the BCE-Dice upgrade

For teammate use, the most important advice is:

- use Colab GPU
- keep `num\_workers = 0`
- expect `/content` files to disappear after resets
- resume from Step 2 onward if the baseline and threshold sweep are already complete